Reason. Plan. Act. Physical AI Workshop

Lab 3.2: Task-then-motion planning

Guy Azran and Sarah Keren

The Taub Faculty of Computer Science
Technion - Israel Institute of Technology



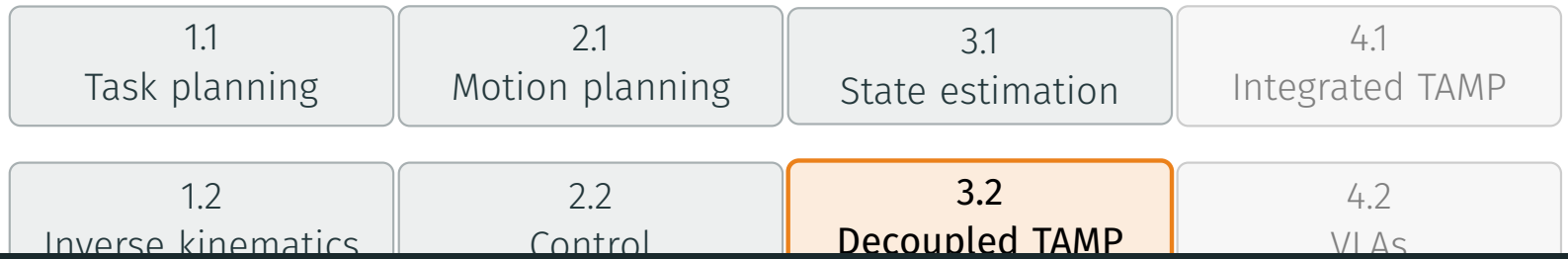
Putting the pieces together

Guy Azran and
Sarah Keren

Five labs gave us every piece of a robot that reasons, plans and acts:

lab	from	to
1.1 Task planning	a PDDL problem	a sequence of actions
1.2 Inverse kinematics	a tool pose	joint angles
2.1 Motion planning	a configuration	another one, around obstacles
2.2 Control	a target	torques that track it
3.1 State estimation	the world	facts for the task planner

Today: the simplest way to connect them. Task planning *then* motion planning.

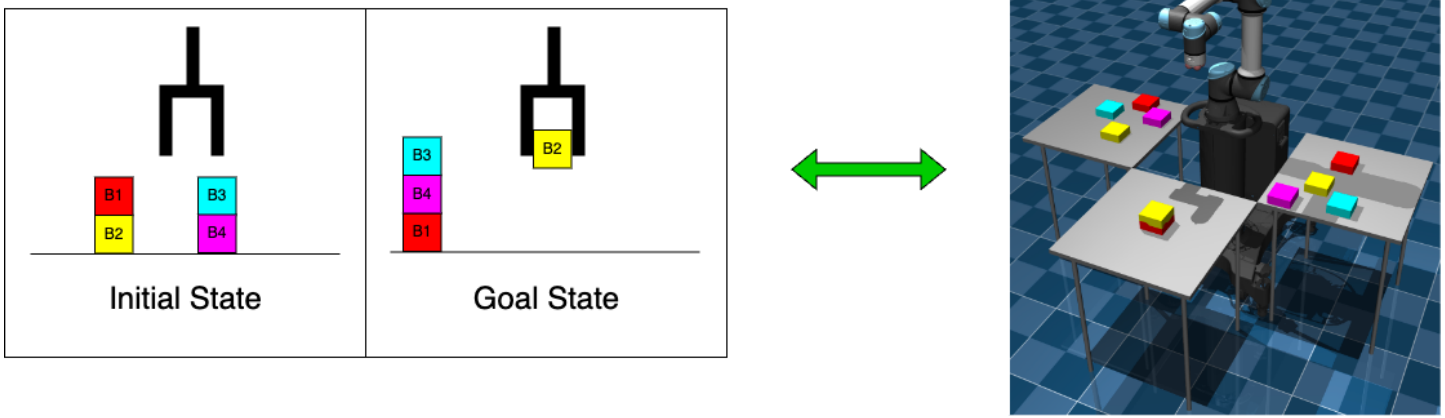


Today

Learn	Skills: from actions to motions
Learn	Sampling where to put a block down
Do	Task 1: sample a free spot on a table
Do	Task 2: from an action to a skill
Do	Task 3: execute a plan, open loop
Learn	Three ways for a plan to fail
Do	Task 4: what the planner expects
Learn	Monitoring and replanning
Do	Task 5: execute with monitoring
Learn	The limits of decoupling
Do	Bonus: the camera instead of the oracle

Read the plan as if you were the robot

Guy Azran and
Sarah Keren




```
unstack(b2, b1)
put-down(b2, t1)
pick-up(b3, t2)
stack(b3, b1)
```

how high above `b2`? from which side?

where on `t1`, a table behind a wall?

what if `b1` moved since the plan was made?

The plan leaves this out on purpose: it keeps the planning problem small. We fill it in.

Two worlds, two mappings

Guy Azran and
Sarah Keren

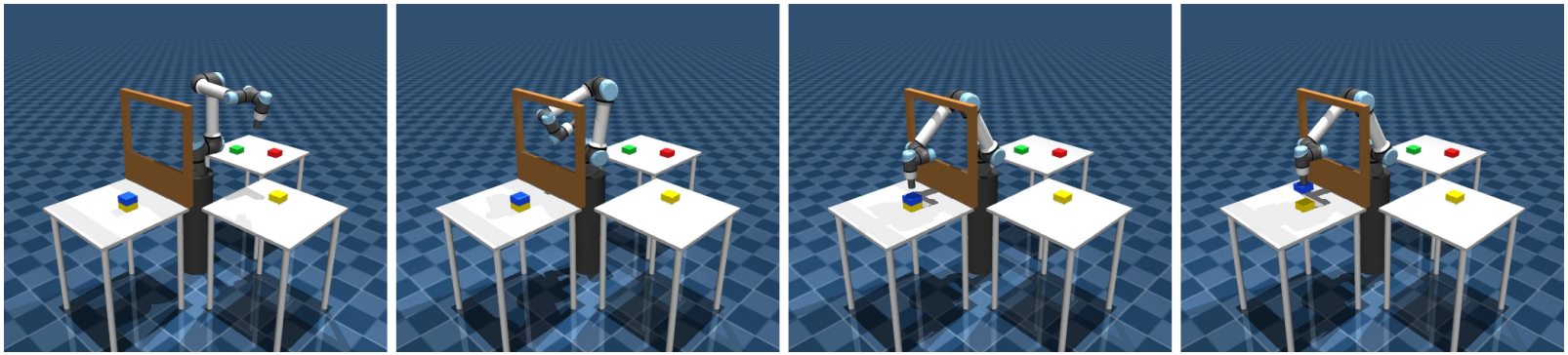


The task planner speaks in objects, facts and actions. The motion planner speaks in configurations.

- **State estimator**: world $\rightarrow$ facts. Before planning, after every action, at the end.
  (Today: the true state, read from the simulator's contacts)
- **Skills**: action $\rightarrow$ motions and gripper commands
- And the names: the planner's `b2` must be the simulator's `b2`

Skills: from actions to motions

Guy Azran and
Sarah Keren



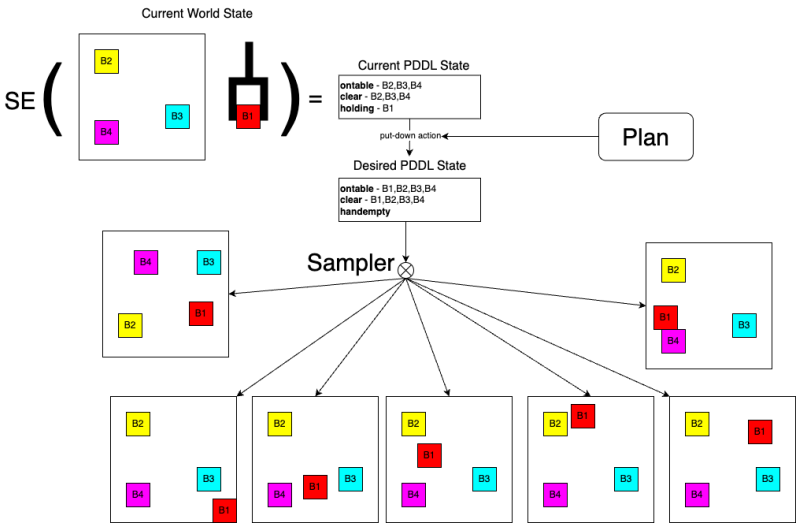
`pick-up`: **approach** (planned, through the window), **descend** (straight down), **grasp**, **lift**

Two changes from Lab 2.1, needed for long plans:

- Choose the grasp configuration first, then the approach configuration in the *same* IK family
- A skill reports failure and stops: no descent after a failed approach

Where to put a block down? The sampler

Guy Azran and
Sarah Keren



`(ontable b2 t1)` holds in infinitely many worlds. The **sampler** picks one, changing only what the action changes.

A good spot is

- on the table
- free
- reachable
- not in the way of what comes next

Rejection sampling: draw a random spot, test, draw again.

Task 1: sample a free spot on a table

Guy Azran and
Sarah Keren

⚒ **Task 1**: Write `sample_on_table(table)`, which returns a uniformly random (x, y) on the top of the table, far enough from the edges for a block centered there to sit fully on it (use `np.random.uniform`). Then `is_free(xy, exclude)`, which is `True` if a block put down at `xy` would overlap none of the blocks currently on the tables.

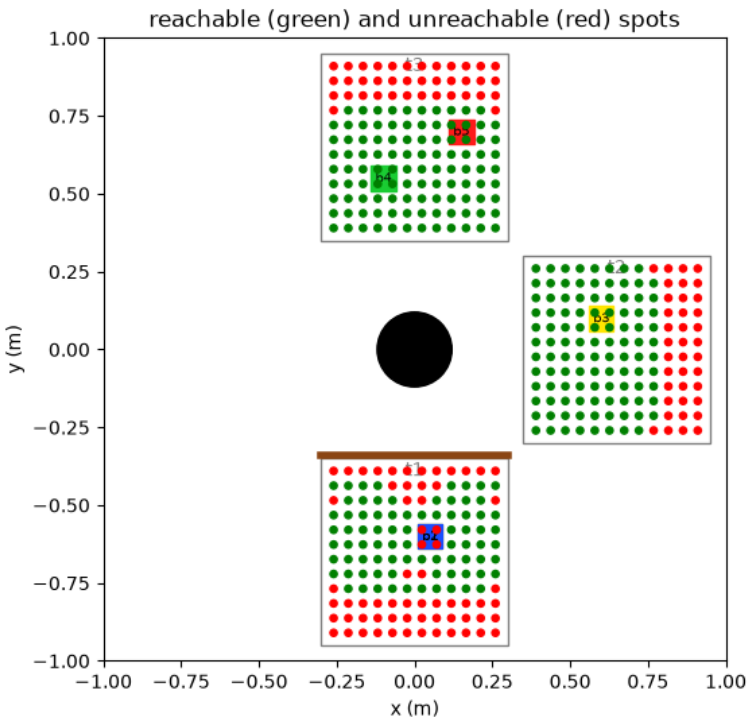
```python
def sample_on_table(table):
    """A uniformly random (x, y) on the table, far enough from the
edges for a block to sit fully on it."""
    _, center, half_size = table_surface(table)
    return center  # your code here
```

```python
def is_free(xy, exclude=None):
    """Whether a block put down at (x, y) would overlap none of
the blocks on the tables (with a safety margin)."""
    return True  # your code here
```

Two blocks overlap if their centers are closer than a block width (plus `MARGIN`) along *both* axes.

Reachable spots

Guy Azran and
Sarah Keren



Guess first: where can the arm not put a block?

- beyond 80 cm from the base
- the strip of `t1` right behind the wall

Reachability is computed by the motion layer (IK and collision checks). It does not belong in the PDDL model; the sampler asks on the task planner's behalf.

Task 2: from an action to a skill

Guy Azran and
Sarah Keren

⚒ **Task 2**: Write `execute_action(action, sampler)`, which runs the skill that implements one action instance of the plan and returns what the skill returns, `(success, frames)`. Read the action's name and its parameters (as strings), and dispatch: `pick-up` and `unstack` both call `pick_up` with the block; `stack` calls `stack`; `put-down` asks the `sampler` for a spot on the table (and fails, with a message, if it returns `None`) and calls `put_down`.

```python
def execute_action(action, sampler=sample_free_spot):
    """Run the skill that implements one action instance of the
task planner. Returns (success, frames)."""
    name, args = action.action.name, [str(p) for p in
action.actual_parameters]
    return False, []  # your code here
```

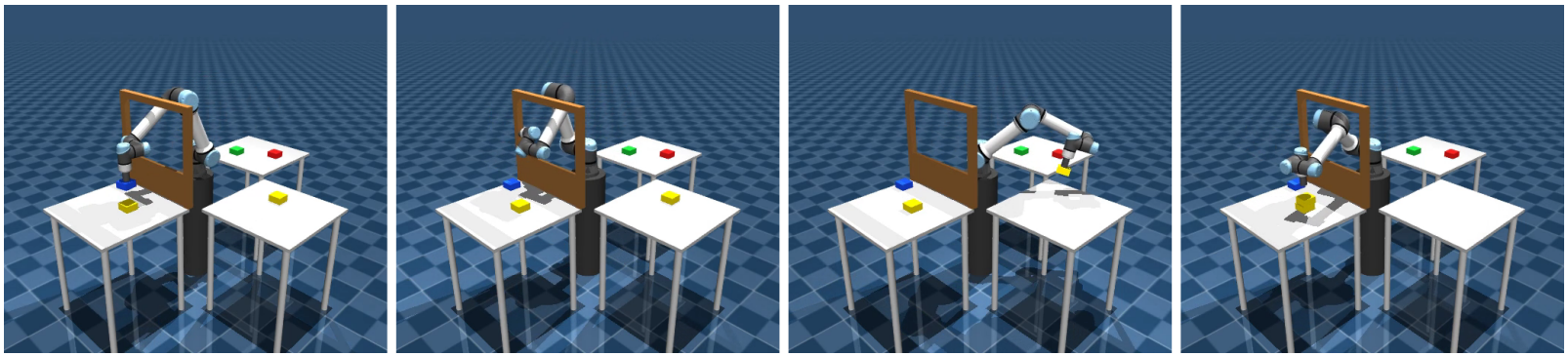
Which parameters does each skill really need?

Task 3: execute a plan, open loop

Guy Azran and
Sarah Keren

⚒ **Task 3**: Write `execute_plan(task_plan, sampler)`, which executes the actions one after the other, collecting the frames, and stops at the first action whose skill fails. Return `(success, frames)`. Then run the plan of tutorial 1.1 from the initial state, watch it, and check whether the goal holds at the end (the goal is a set of facts, and it holds if it is a subset of the estimated state).

```python
def execute_plan(task_plan, sampler=sample_free_spot):
    """Execute the actions of a plan in order, stopping at the
first failure. Returns (success, frames)."""
    frames = []
    # your code here
    return True, frames
```

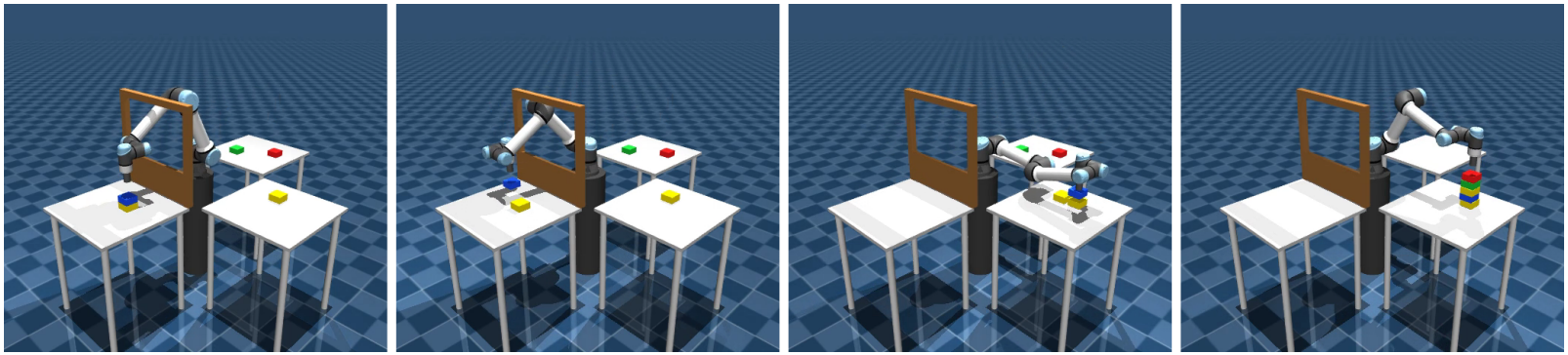


A longer task: a tower of five blocks

Guy Azran and
Sarah Keren

planner	actions
`fast-downward` (satisficing)	26
`fast-downward-opt` (optimal)	12

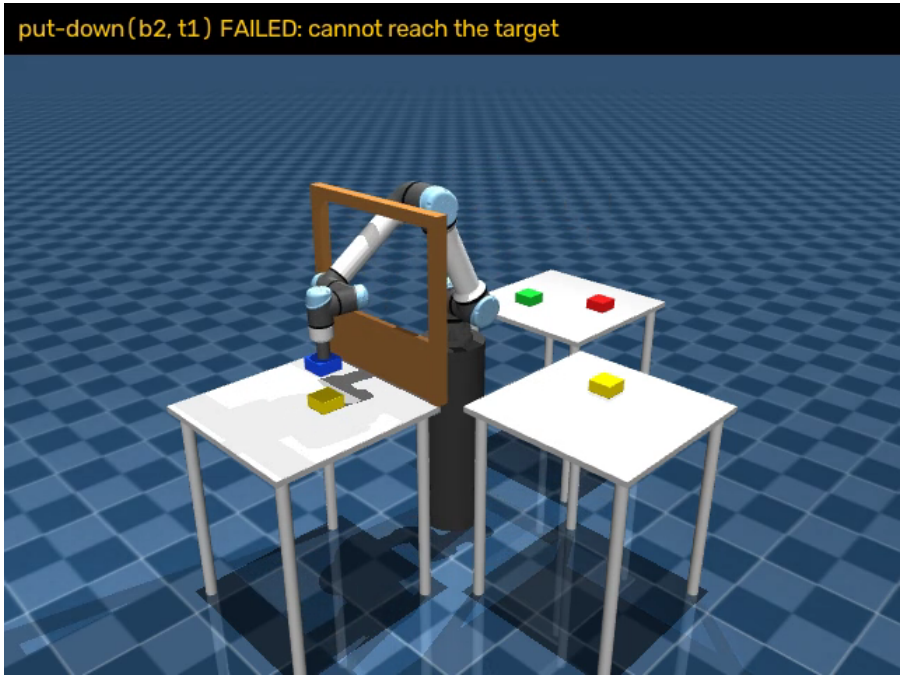
Every action is about ten seconds of motion, and one more chance to fail.

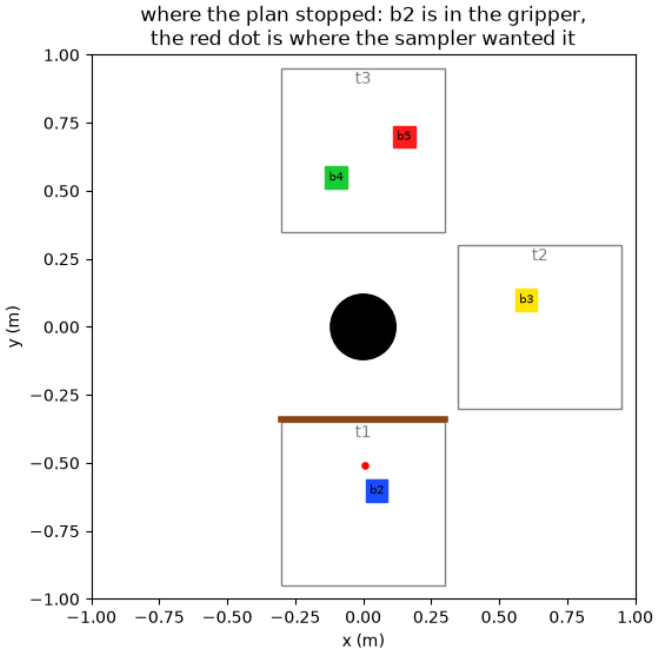


Look at the second frame. The planner parks **b2** behind the wall. It counts actions, so all tables look alike.

When things go wrong (1): the skill knows it failed

Guy Azran and
Sarah Keren



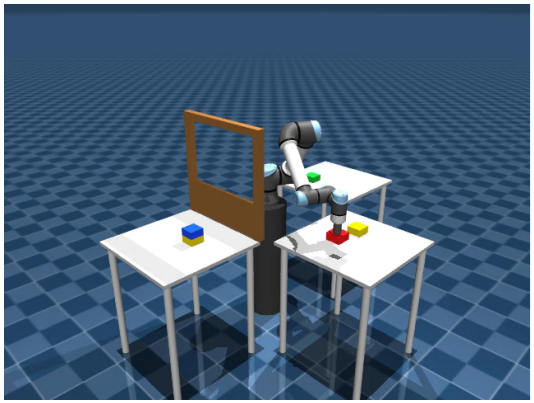


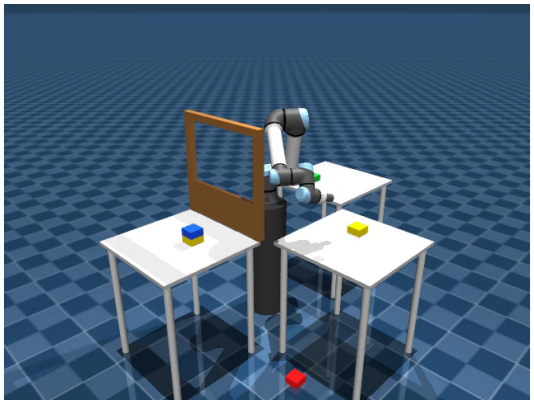
A *naive* sampler proposes a spot behind the wall (red dot). The skill refuses before it moves, and the robot keeps holding **b2**.

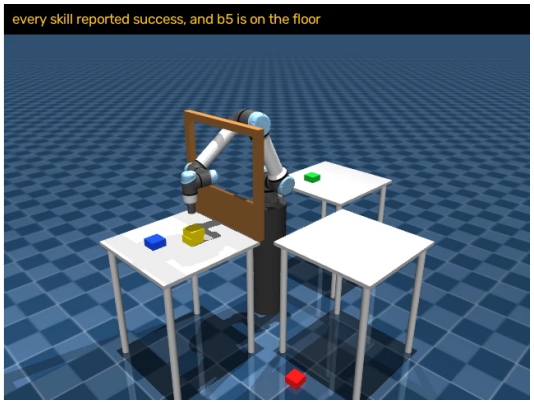
The state is consistent: a new sample would probably work.

When things go wrong (2): the skill thinks it succeeded

Guy Azran and
Sarah Keren





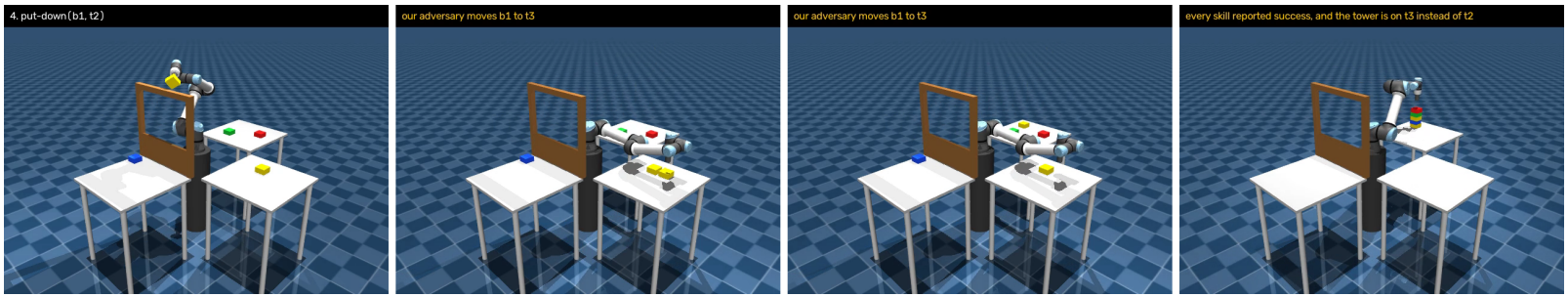


A *careless* sampler with a sign bug: a spot half a block beyond the edge of `t2`.

- All six skills report **ok**; the goal is not achieved
- Skills check their *motion*, not the *effect* of the action
- A skill can only check for the failures its author thought of

When things go wrong (3): someone else changes the world

Guy Azran and
Sarah Keren



After the fourth action of the tower, an **adversary** moves `b1` to `t3`.

- No skill failed: all twelve report **ok**
- The *plan* stopped being valid, and nothing noticed

How can the robot notice earlier? Compare what *is* (the estimator) with what *should be* (the planner).

Task 4: what the planner expects

Guy Azran and
Sarah Keren

⚒ **Task 4**: Write `expected_states(problem, task_plan)`, which returns a list with the set of facts the planner expects after each action of the plan, using `SequentialSimulator` (from tutorial 1.1: `simulator.get_initial_state()`, `simulator.apply(state, action)`) and `facts_of`.

```python
def expected_states(problem, task_plan):
    """The facts that the task planner expects to hold after each action of the plan (a list of sets)."""
    states = []
    # your code here
    return states
```

- Apply the plan's actions one by one, as in Lab 1.1
- One set of facts per action

Closing the loop: monitor and replan

1. Estimate the state. If the goal holds, stop.
2. Plan from the *estimated* state.
3. Execute action by action. After each one, estimate again and compare with the expected state.
4. If a skill failed or the states differ, go back to 1.

Also called **execution monitoring** or **dynamic replanning**.

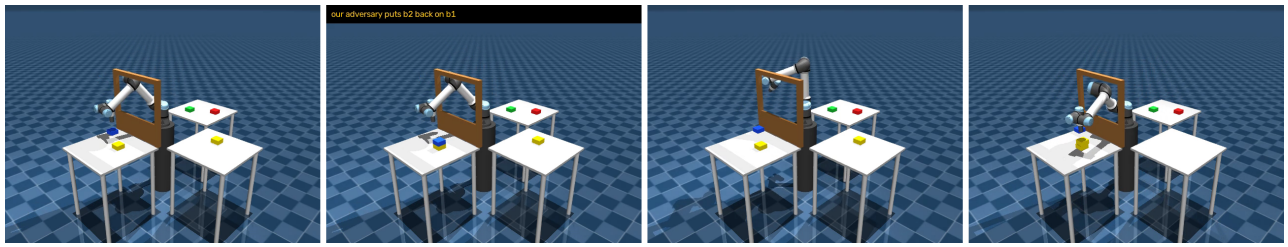
It needs no model of what can go wrong: an adversary, a slipping block and a toppled stack are handled the same way.

Task 5: execute with monitoring

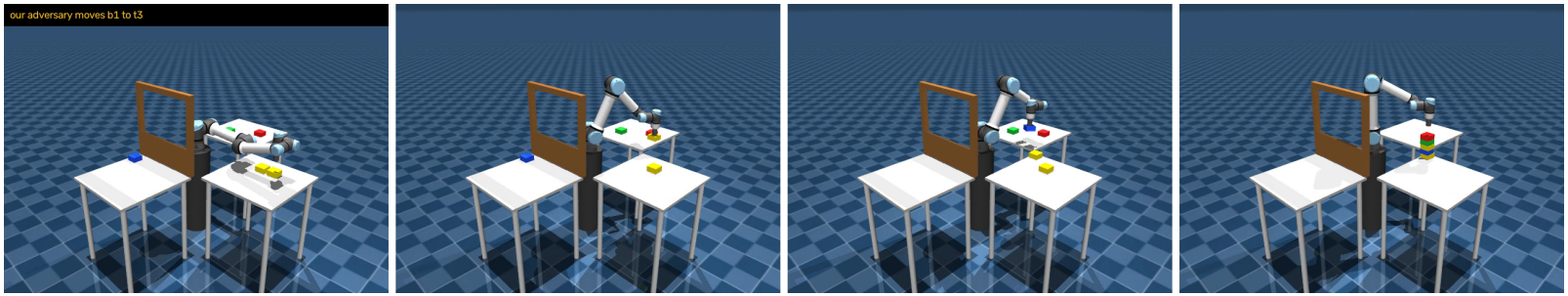
Guy Azran and
Sarah Keren

⚒ **Task 5**: Complete `execute_with_monitoring(goal, ...)` following the steps above. Print what happens: each plan, each action, and the differences between the expected and the estimated state when they disagree. Return `(success, frames)`.

```
def execute_with_monitoring(goal, max_replans=5,
sampler=sample_free_spot, after_action=None):
    """Plan, execute, check after every action, and replan when
needed. Returns (success, frames)."""
    frames = []
    steps = 0  # actions executed so far
    for attempt in range(max_replans + 1):
        facts = estimate_state()
        # your code here: check the goal, plan, execute action by
action while the expected state matches
    return False, frames
```



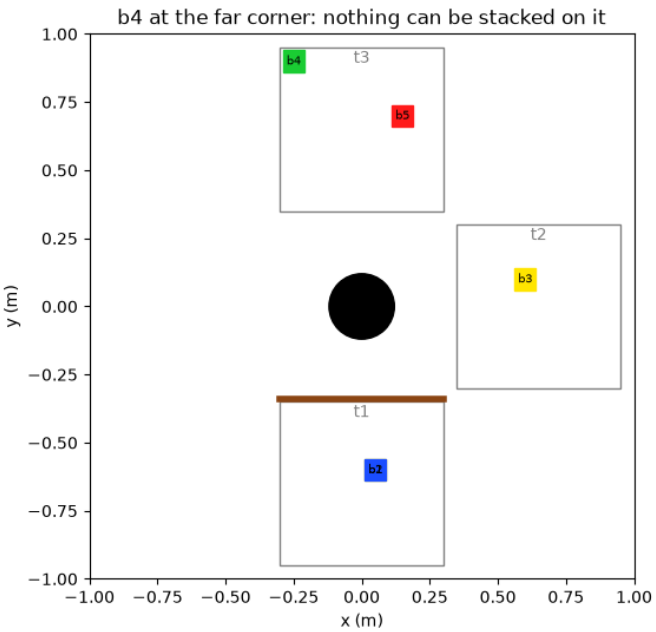
The closed loop at work



- **The tower with the adversary**: one replan, 14 actions in total, the tower stands on `t2`
- **The naive sampler**: the skill refuses, the robot replans, a new sample works
- **The careless sampler**: the monitor misses `(ontable b5 t2)` and replans.
  The planner finds no plan, and since it is *complete*, that is a proof: the robot stops and can tell a person.

The limits of decoupling

Guy Azran and Sarah Keren



`b4` is out of reach. Goal: `(on b5 b4)`.

The skill fails, nothing changes, and the planner proposes the same plan again and again.

Replanning fixes failures whose cause goes away, not failures that repeat.

The task planner does not know what the skills know:

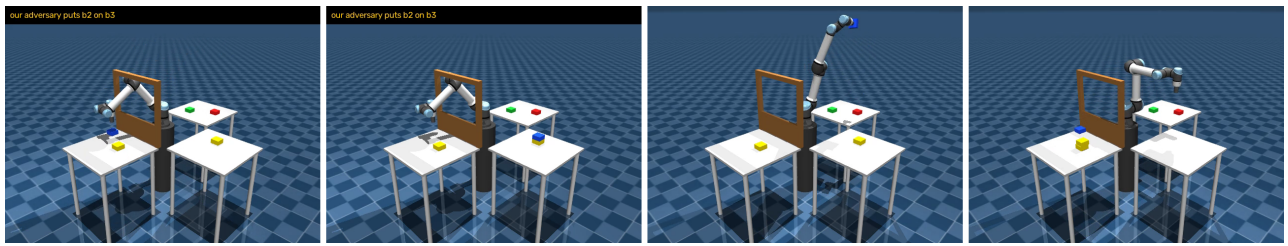
- **infeasible actions**: `(clear b4)` holds, yet stacking fails
- **blind choices**: all tables cost the same
- **wasted effort**: failures are found at the speed of a robot arm

Bonus: the camera instead of the oracle

Guy Azran and
Sarah Keren

⚒ **Bonus task**: Replace `estimate_state` by one that looks through the camera and returns `abstract_state(positions, held)`. The estimator needs the initial positions once (as in tutorial 3.1, we hand it the true ones), so create it right after `reset`. Then run the first task again with the adversary, who this time puts `b2` on top of `b3`, and compare the robot's account of the world with the truth at the end.

```
def estimate_state():
    """The abstract state of the world, as the camera sees it."""
    settle()
    return true_state()  # your code here: look, update the
estimator, and return the abstract state
```



Hint: from above, the brown window frame looks yellow.

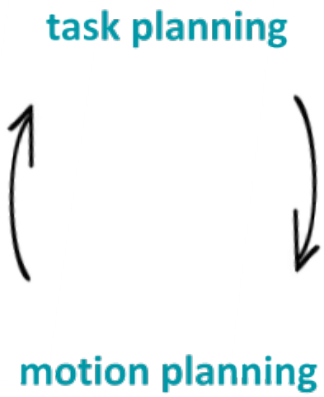
Get to work!

Summary

In this tutorial we

- connected symbols and numbers with two mappings: the state estimator (world $\rightarrow$ facts) and skills (action $\rightarrow$ motions)
- wrote a sampler that decides *where*: free, reachable spots on a table
- executed plans open loop, and saw three ways they fail
- closed the loop: monitored every action, and replanned when the world disagreed with the plan
- built a tower of five blocks while an adversary moved them
- found failures that replanning cannot fix: the task planner does not know what the skills know

What comes next



Today the information flowed one way: from the task planner to the motion layer.

Integrated TAMP (Lab 4.1): the motion layer tells the task planner

- where there is room
- what motions really cost
- which actions cannot be done, found in simulation before the robot moves